

CODEBASE HANDOVER

How the Platform *Works*

A plain-English walkthrough of the document ingestion and analysis pipeline — what happens to a deal document from the moment it is uploaded, and where each step lives in the code.

PRODUCT

Claude-native M&A platform

AUDIENCE

Engineering handover

SCOPE

Backend pipeline

What's inside

This document follows a single deal file on its journey through the system. You can read it start-to-finish, or jump to any part. No coding knowledge required — technical file references are included in the side so an engineer can trace anything back to the source.

01 What DealDiligence.ai is p.3

The product in one paragraph, and the problem it solves

02 The two big ideas p.4

The design choices that make everything else simple

03 The journey of a document p.5

The whole pipeline on one page

04 Stage by stage p.6

Upload · Reading · Verifying · Saving, explained simply

05 Connecting the dots p.9

How many documents become one living "deal brief"

06 Putting it to work p.10

The AI task board and the ask-anything chat

07 Who sees what p.11

How access control protects sensitive folders

08 What it costs to run p.12

Real numbers per deal, and the margin story

09 Where everything lives & glossary p.13

A map of the code, and a jargon decoder

What DealDiligence.ai is

When companies buy, sell, or merge, lawyers and bankers must read **hundreds of dense contracts** to find the risks hidden inside them. That review — called *due diligence* — is slow, expensive, and easy to get wrong.

DealDiligence.ai is a secure workspace where a deal team uploads those contracts and an AI assistant — **Claude** — reads every page end-to-end. For each document it produces a clean one-page summary: who the parties are, what the key clauses say, a **risk score out of 10**, and the biggest red flags. It then stitches every document together into a single living summary of the whole deal, answers questions in plain English, and drafts risk memos on request — all while making sure a junior reviewer can only ever see the folders they've been granted.

/10

Every document gets a risk score, measured against the client's own playbook

2^x

Independent checks catch the AI's mistakes before a human trusts the output

1

Living "deal brief" feeds the chat, the task board, and the dashboard

Who uses it

Investment bankers, M&A lawyers, corporate-development teams, private-equity and venture investors, and auditors — anyone who sits on one side of a deal and needs to understand a mountain of paperwork quickly and safely.

THE ONE-LINE PITCH

Think of it as a tireless junior analyst that reads every contract overnight, never misses a clause, shows its work with citations, and hands the senior team a ranked list of what to worry about.

The two big ideas

Almost every clever thing in this codebase comes from two simple decisions. Understanding them makes the rest obvious.

① Everything downstream reads a summary, never the raw PDF

A PDF is heavy and expensive for an AI to read. So the system reads each PDF **once**, very carefully, and turns it into a lightweight text summary called a **fact sheet**. After that, every other feature — the chat, the dashboard, the risk memos — reads those tidy fact sheets instead of re-reading the original files. That one choice makes the whole platform fast, cheap, and easy to control.

② Access control is just "which summaries can you load?"

Because everything reads summaries, restricting what a person can see becomes simple: you decide **which fact sheets get handed to the AI** for that person. A junior reviewer limited to the "Financials" folder literally never has the other documents placed in front of the AI on their behalf — so the chat, their dashboard, and their reports can only ever draw on what they're allowed to see.

WHY THIS MATTERS FOR THE BUSINESS

These two ideas are why the product is both **cheap to run** (a full deal costs single-digit to low-hundreds of dollars in AI fees) and **safe to sell** to regulated clients (information barriers are enforced by design, not by trust).

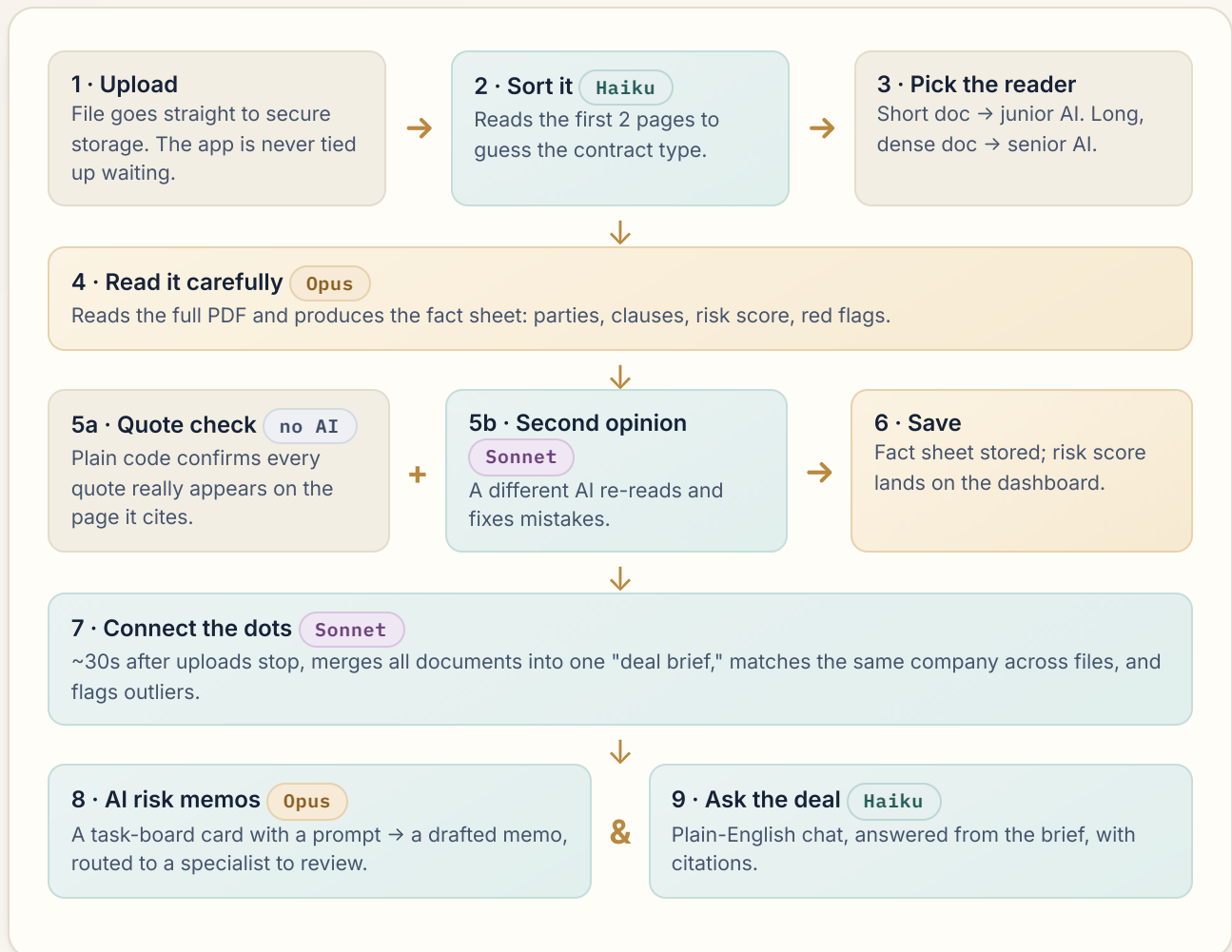
A note on the AI itself

The platform uses three "sizes" of Claude, matched to the job — the same way you'd assign work to staff of different seniority. This keeps quality high where it counts and costs low everywhere else.

MODEL	THINK OF IT AS	USED FOR
Haiku 4.5	Fast junior	Quick sorting of documents, and the high-volume Q&A chat
Sonnet 4.6	Capable associate	Double-checking the AI's work, and merging documents together

The journey of a document

Here is the entire pipeline on one page. The following sections walk each block in plain English. Gold blocks are the careful reads; teal blocks are AI working in the background.



THE SHAPE TO REMEMBER

One careful read per document produces a **fact sheet** (artifact #1). All the fact sheets get merged into a **deal brief** (artifact #2). Every feature you can click on is just reading one of those two artifacts.

Stage by stage

1

Upload — the handshake before any AI

When a user adds a file, the browser uploads it **directly to secure cloud storage** (Amazon S3) using a temporary one-time link. The bytes never pass through our own servers, and the app responds instantly — it doesn't sit and wait for the AI. A background job is kicked off, and the screen simply shows *Uploading* → *Processing* → *Complete* as it progresses.

WHY IT'S BUILT THIS WAY

The expensive AI work never blocks the website. A user can upload 50 files and keep working while they process in the background.

In the code:

```
modules/documents/documents.service.ts
```

— `initiateUpload()` hands out the upload link; `confirmUpload()` starts the background read.

2

Sort it Haiku

Before the careful read, the fast junior AI skims just the **first two pages** to decide what kind of document this is — a purchase agreement, an NDA, a lease, a financial statement, and so on (nine types in all). The document type is almost always obvious from the cover page, so there's no need to pay to read all 80 pages here.

```
integrations/claude/classify.ts
```

3

Pick the right reader — the cost lever

Now the system chooses which "size" of AI does the heavy read, based on length and type. A 3-page NDA goes to the cheap junior; a 200-page credit agreement goes to the senior expert. There's one smart override: term-dense documents like purchase agreements are always bumped up a level, even when short — they're too important to trust to the junior.

THIS IS THE DIAL THAT CONTROLS COST

Reading every document with the most expensive AI would work — it would just cost several times more. Matching the model to the job is what keeps a full deal in the tens of dollars.

```
integrations/claude/model-router.ts — pickExtractionModel()
```

4

The careful read Opus

This is the heart of the product. The chosen AI reads the **entire PDF** and returns a structured **fact sheet**: an executive summary, the parties involved, the important clauses (organised by a legal-industry standard called CUAD), a **risk score out of 10**, the top risks, key dates and dollar amounts, and the relationships between them.

Two refinements make the result better and cheaper:

- **The client's playbook is included.** If a client has told us their preferred positions and red flags, the risk score measures how far this contract *deviates from what they consider acceptable* — not just generic risk.
- **The instructions are "cached."** The lengthy rulebook the AI follows is the same every time, so the system stores it and reuses it at a 90% discount on repeat reads.

```
integrations/claude/extract.ts
```

```
.../prompts/extraction/
```

5

The two safety nets — the defining feature

An AI that reads contracts is only useful if you can **trust** what it says. So before any result is shown, it passes two independent checks:

Net 1 • The quote checker no AI

Plain, deterministic code pulls the real text out of the PDF and confirms every quote the AI used **actually appears on the page it claims**. If a quote is on a different page, it's flagged. If a quote doesn't appear anywhere, it's marked a likely fabrication. No AI can talk its way past this — it's pure math.

Net 2 • The second opinion

Sonnet

A **different** AI re-reads the original PDF alongside the fact sheet, hunting for made-up quotes, wrong pages, missing clauses, or a risk score that doesn't match the text. Crucially, it can hand back a **corrected version**, which the system saves in place.

WHAT THE CLIENT SEES

Each document ends up marked **Verified**, **Needs review**, or **Failed** — so a human always knows how much to trust it. This "show your work" guarantee is the product's biggest differentiator.

```
utils/citation-validator.ts
```

```
integrations/claude/verify.ts
```

6

Save the result

The finished fact sheet is written to cloud storage, and the key facts are copied into the database so the rest of the app can sort and filter instantly — the risk score lands on the dashboard, the document type becomes a filter, the parties and dates become searchable. The full write-up stays as the durable, human-readable record.

A QUIET MONEY-SAVER

Each file is fingerprinted. If the exact same file is uploaded again unchanged, the system recognises it and **skips the expensive re-read entirely** — re-uploading a file is free.

```
services/extraction.service.ts
```

— the conductor that runs steps 2–6 in order, with automatic retries if anything fails.

A note on reliability

Steps 2 through 6 are wrapped in a single coordinator that handles the messy realities of working with an outside AI service: if a request is rate-limited, it waits and retries; if something fails, it backs off and tries again up to three times before marking the document as failed (which a user can manually retry). And if no AI is configured at all — for a demo or offline development — the whole flow still runs on canned sample data, so the product is always clickable.

In one sentence

A document comes in, gets sorted, gets read by the right-sized AI, gets checked twice, and comes out the other side as a trustworthy one-page summary with a risk score — all in the background, without anyone waiting at a screen.

Connecting the dots

One document is useful. The real value is understanding the **whole deal** — and that means tying every document together.

7

The deal brief Sonnet

About **30 seconds after the last upload finishes**, a background pass reads every fact sheet in the deal and produces a single, living **deal brief**: the parties, the key clauses across all documents, the top five risks for the whole deal, the important dates, and how the documents relate. This brief is the one summary the chat, the task board, and the dashboard all read from.

WHY THE 30-SECOND WAIT

If someone bulk-uploads 20 files, the system doesn't rebuild the brief 20 times. It waits until the dust settles and rebuilds **once** — a simple trick that keeps costs down during busy uploads.

It also matches duplicates

"Acme Corp" in one file and "Acme Corporation" in another are recognised as the **same company** and merged into one master record — so the deal isn't cluttered with phantom duplicate parties. This also powers a visual relationship map for the team.

And it flags outliers

Once there are at least three comparable documents, the AI can spot when one contract's terms **deviate from its peers** — an unusually long liability cap, an off-market interest rate — and flag it for attention.

THE SUBTLE PART: MANY BRIEFS, NOT ONE

A junior reviewer restricted to certain folders must not see a brief that summarises documents they're not allowed to view. So the system quietly builds a **separate brief for each distinct access level** in use. Two people with the same folder access share one brief; a restricted reviewer gets their own, built only from their documents. Their chat, dashboard, and reports all draw from that limited brief automatically.

```
services/reconciliation.service.ts
```

```
services/deal-brief.service.ts
```

```
utils/scope-key.ts
```

One more thoughtful touch: the deal team's hand-written notes inside a brief are preserved *word-for-word* every time the AI regenerates it — human edits are never overwritten.

Putting it to work

With trustworthy summaries in place, two features turn them into real work product.

8

The AI task board Opus

The platform has a Kanban board — the familiar columns of cards you drag from *To-do* to *Done*. Here, a card can carry an **instruction** ("Assess the change-of-control provisions") and a few **attached documents**. When a team member drags it into progress, the senior AI reads the deal brief plus those documents and **drafts a risk memo**. The card then moves itself into the *In Review* column, where a specialist checks the AI's work.

THE WORKFLOW INTENT

The AI drafts; the human approves. The card literally walks itself from "to-do" to "ready for a specialist" — the AI becomes a member of the team, not a black box.

A safety check runs first: if the person isn't allowed to see an attached document's folder, the report is refused — they can't use the AI to peek at restricted files.

```
modules/tasks/task-ai.service.ts
```

9

Ask the deal anything Haiku

Inside the data room, anyone on the team can ask questions in plain English — "*What's the longest non-compete in these contracts?*" The fast junior AI answers from the user's scoped deal brief and returns **citations** (which document, which page) so the answer is checkable. If a person has been granted no folders yet, the chat politely declines rather than guessing — access is enforced even here.

```
modules/chat/chat.service.ts
```

The common thread: notice that both features read the *same* deal brief and fact sheets — never the raw PDFs. That's the first big idea paying off. Adding a new AI feature means writing a new prompt over the same summaries, not building new plumbing.

Who sees what

M&A deals are full of information barriers — a junior analyst or an outside party may only be cleared for part of the data room. The platform enforces this everywhere, from one central rulebook.

The seniority ladder

Every member of a deal has a role. Owners and admins see everything. More junior members get an explicit list of folders they're allowed into — and granting a folder automatically grants everything inside it.

ROLE	WHAT THEY CAN SEE
Owner / Admin	The entire deal — full access by default
Member / Viewer	Only their granted folders (and any sub-folders inside them)
Super Admin / Company Admin	Oversight across their organisation's deals

THE SINGLE CHOKEPOINT

One function answers the question "what is this person allowed to see?" and **every feature** — documents, dashboard, chat, AI reports — asks it before doing anything. Access rules live in one place, not scattered across the app, which is exactly what an auditor wants to hear.

Enforced at every door

- Document lists hide what you can't open
- The dashboard only totals your folders
- Chat retrieval is filtered to your scope
- AI reports refuse out-of-scope attachments

Where it lives

```
services/scope.service.ts
```

```
middleware/permissions.ts
```

```
utils/scope-key.ts
```

TYING IT BACK

Remember big idea #2: because access is "which summaries get loaded," these rules are a simple filter on a list — not a fragile maze of special cases. That's why the security story is strong enough to take to a regulated buyer.

What it costs to run

Because the design reads each PDF only once and reuses cheap summaries everywhere else, the AI cost of a deal is remarkably low.

DEAL SIZE	DOCUMENTS	AI COST / DEAL
Small	10 documents	≈ \$8
Mid-size	25 documents	≈ \$32
Large	80 documents	≈ \$150
Jumbo	200 documents	≈ \$433

AI usage fees only. The careful first read is the main cost; chat is effectively free at about half a cent per message.

The margin story

At a representative price of **\$500 per user per month**, a five-person team running three mid-size deals a month produces healthy economics:

\$2,500 Monthly revenue (5 seats)	~\$500 AI + infrastructure cost	~80% Gross margin
---	---	-----------------------------

COST IS A DIAL, NOT A WORRY

Several levers can cut costs further if ever needed — downgrading the heaviest read to the mid-tier AI saves roughly 5× on the biggest line item, batch processing offers ~50% off, and skipping the merge step for tiny deals. None are needed today; they're headroom.

Full breakdown, token math, and infrastructure figures live in `COST_ANALYSIS.md` at the project root.

Where everything lives

A quick map for the engineer taking this over. Everything is one Node/TypeScript backend — no separate AI service, no extra databases.

WHAT IT DOES	FILE
Upload handshake	<code>modules/documents/documents.service.ts</code>
The read pipeline (conductor)	<code>services/extraction.service.ts</code>
Sort · route · read · verify	<code>integrations/claude/</code> <code>classify</code> · <code>model-router</code> · <code>extract</code> · <code>verify</code>
The quote checker (no AI)	<code>utils/citation-validator.ts</code>
Merge documents + flag outliers	<code>services/reconciliation.service.ts</code>
The living deal brief	<code>services/deal-brief.service.ts</code>
AI task-board memos	<code>modules/tasks/task-ai.service.ts</code>
Ask-the-deal chat	<code>modules/chat/chat.service.ts</code>
Who-sees-what rules	<code>services/scope.service.ts</code>
The database shape	<code>prisma/schema.prisma</code>

Jargon decoder

Fact sheet

The one-page AI summary of a single document. Artifact #1.

Deal brief

One living summary of the whole deal, built from all fact sheets. Artifact #2.

Claude

The AI model the platform runs on, in three sizes (Haiku, Sonnet, Opus).

S3

Amazon's secure cloud file storage, where PDFs and summaries are kept.

CUAD

A legal-industry standard list of contract clause types, used so extraction is consistent.

Playbook

A client's own preferred positions and red flags; risk is scored against it.

Prompt caching

DEALDILIGENCE.AI — CODEBASE WALKTHROUGH
Reusing the AI's unchanging instructions at a 90% discount.

Scope

The set of folders a person is allowed to see; everything filters by it.